

Chapter 12

Image pattern classification

IMP302m

Contents

- 12.1 Patterns and pattern classes
- 12.2 Pattern classification by prototype matching
- 12.3 Optimum statistical classifier
- 12.4 Neural network and Deep learning based classifiers
- 12.5 Deep convolutional neural network based classifiers

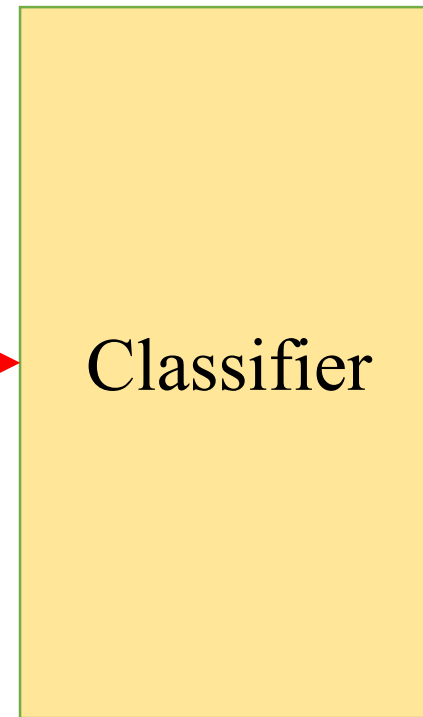
Fundamentals

- In image pattern recognition, a *pattern* as a spatial arrangement of features
- A *pattern class* is a set of patterns that share some common properties
- Pattern recognition by machine encompasses techniques for automatically assigning patterns to their respective classes
- There are four main stages involved in recognition:
 - Sensing
 - Preprocessing
 - Feature extraction
 - Classification

Fundamentals

- Sensing generates signals in a spatial (2-D) or higher-dimensional format
- Preprocessing deals with techniques for tasks such as noise reduction, enhancement, restoration, and segmentation
- Feature extraction: extracts the significant information from raw images
- Classification: assigns a label or category to an input data point based on its features

Fundamentals

 c_1 c_2

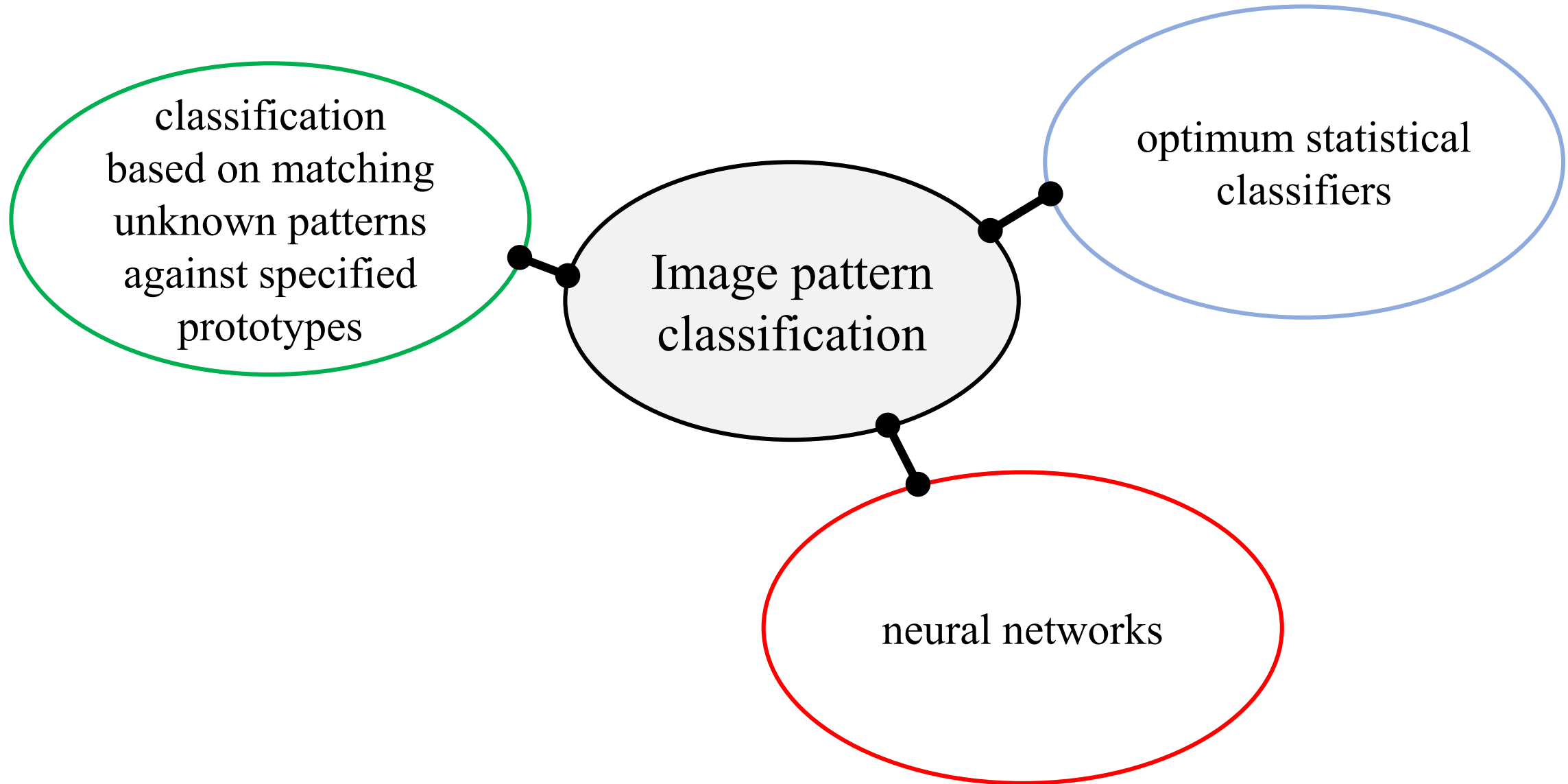
$X =$

125	27	202	25	57
27	129	132	150	93
157	198	160	233	254
13	15	72	45	54
18	29	73	100	150

$$\mathbf{x} = \begin{bmatrix} 230 \\ 240 \\ 15 \\ \vdots \\ 150 \end{bmatrix}$$

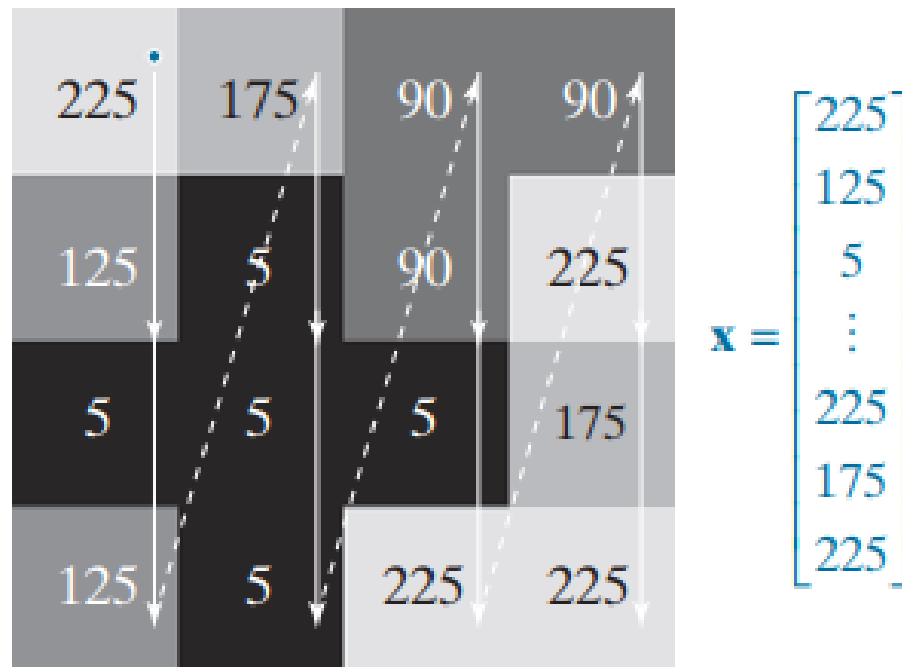
$$\mathbf{y} = \begin{bmatrix} 0.83 \\ 0.17 \end{bmatrix}$$

Fundamentals



12.1 Patterns and pattern classes

- Pattern vectors $\mathbf{x} = [x_1 \ x_2 \ \dots \ x_n]^T$ where x_i represents the i -th feature descriptor and n is the total number of such descriptors



12.1 Patterns and pattern classes

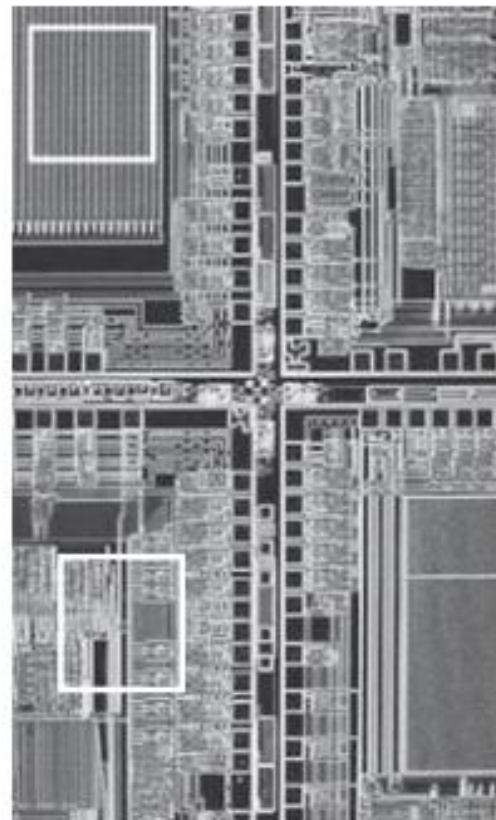
Structural patterns

- Pattern vectors are not suitable for applications in which objects are represented by structural features, such as strings of symbols



$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

x_1 = compactness
 x_2 = circularity
 x_3 = eccentricity



$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix}$$

x_1 = max probability
 x_2 = correlation
 x_3 = contrast
 x_4 = uniformity
 x_5 = homogeneity
 x_6 = entropy

12.2 Pattern classification by prototype matching – Minimum-distance classifier

- It computes a distance-based measure between an unknown pattern vector and each of the class prototypes
- It assigns the unknown pattern to the class of its closest prototype

12.2 Pattern classification by prototype matching – Minimum-distance classifier

- The prototype vectors of the minimum-distance classifier are the mean vectors of the various pattern classes

$$\mathbf{m}_j = \frac{1}{n_j} \sum_{\mathbf{x} \in C_j} \mathbf{x} \quad j = 1, 2, \dots, N_c$$

- If Euclidean distance is used to determine similarity, the minimum-distance classifier computes the distances

$$D_j(\mathbf{x}) = \|\mathbf{x} - \mathbf{m}_j\| \quad j = 1, 2, \dots, N_c$$

- An unknown pattern $\mathbf{x}$ is assigned to class c_i if

$$D_i(\mathbf{x}) < D_j(\mathbf{x}) \text{ for } j = 1, 2, \dots, N_c \text{ and } j \neq i$$

12.2 Pattern classification by prototype matching – Minimum-distance classifier

- Selecting the smallest distance is equivalent to evaluating the decision functions

$$d_j(\mathbf{x}) = \mathbf{m}_j^T \mathbf{x} - \frac{1}{2} \mathbf{m}_j^T \mathbf{m}_j \quad j = 1, 2, \dots, N_c$$

- Assigning an unknown pattern $\mathbf{x}$ to the class c_i whose prototype yielded the largest value of d

$$d_i(\mathbf{x}) > d_j(\mathbf{x}) \quad j = 1, 2, \dots, N_c; \quad j \neq i$$

12.2 Pattern classification by prototype matching – Correlation

- *Correlation coefficient*

$$\gamma(x, y) = \frac{\sum_s \sum_t [w(s, t) - \bar{w}] [f(x + s, y + t) - \bar{f}_{xy}]}{\left\{ \sum_s \sum_t [w(s, t) - \bar{w}]^2 [f(x + s, y + t) - \bar{f}_{xy}]^2 \right\}^{1/2}}$$

- w is often referred to as a *template* (i.e., a prototype subimage) and correlation coefficient is referred to as *template matching*

12.3 Optimum statistical classifier - Derivation of the Bayes classifier

Definitions:

- The probability that a pattern vector $\mathbf{x}$ comes from class c_i is denoted by $p(c_i|\mathbf{x})$
- If the pattern classifier decides that $\mathbf{x}$ came from class c_j when it actually came from c_i it incurs a *loss* (to be defined shortly), denoted by L_{ij}
- Because pattern $\mathbf{x}$ may belong to any one of N_c possible classes, the average loss incurred in assigning $\mathbf{x}$ to class c_j is

$$r_j(\mathbf{x}) = \sum_{k=1}^{N_c} L_{kj} p(c_k|\mathbf{x}) \quad \leftarrow \text{The conditional average risk or loss}$$

12.3 Optimum statistical classifier - Derivation of the Bayes classifier

- Apply Bayes' rule for the loss function

$$r_j(\mathbf{x}) = \frac{1}{p(\mathbf{x})} \sum_{k=1}^{N_c} L_{kj} p(\mathbf{x}|c_k) P(c_k)$$

where $p(\mathbf{x}|c_k)$ is the probability density function (PDF) of the patterns from class c_k and $P(c_k)$ is the probability of occurrence of class

- Because $1/p(\mathbf{x})$ is positive and common, the average loss reduces to

$$r_j(\mathbf{x}) = \sum_{k=1}^{N_c} L_{kj} p(\mathbf{x}|c_k) P(c_k)$$

12.3 Optimum statistical classifier - Derivation of the Bayes classifier

- If the classifier computes $r_1(\mathbf{x})$, $r_2(\mathbf{x})$, $\dots$, $r_{N_c}(\mathbf{x})$ for each pattern $\mathbf{x}$ and assigns the pattern to the class with the smallest loss, the total average loss with respect to all decisions will be minimum $\rightarrow$ The classifier that minimizes the total average loss is called the *Bayes classifier* $\rightarrow \mathbf{x}$ is assigned to class c_i if

$$\sum_{k=1}^{N_c} L_{ki} p(\mathbf{x}|c_k) P(c_k) < \sum_{q=1}^{N_c} L_{qj} p(\mathbf{x}|c_q) P(c_q) \quad \forall j \neq i$$

- A correct decision is assigned a value of 0, and 1 for otherwise. Then, the loss function becomes

$$L_{ij} = 1 - \delta_{ij} \quad \text{where} \quad \delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

12.3 Optimum statistical classifier - Derivation of the Bayes classifier

- The loss function is rewritten

$$r_j(\mathbf{x}) = \sum_{k=1}^{N_c} (1 - \delta_{kj}) p(\mathbf{x}|c_k) P(c_k) = p(\mathbf{x}) - p(\mathbf{x}|c_j) P(c_j)$$

- The condition to assign to classes becomes

$$p(\mathbf{x}) - p(\mathbf{x}|c_i) P(c_i) < p(\mathbf{x}) - p(\mathbf{x}|c_j) P(c_j)$$

$$\Leftrightarrow p(\mathbf{x}|c_i) P(c_i) > p(\mathbf{x}|c_j) P(c_j) \quad j = 1, 2, \dots, N_c; j \neq i$$

- Decision function $d_j(\mathbf{x}) = p(\mathbf{x}|c_j) P(c_j)$ for Bayes classifier for a 0-1 loss function $\rightarrow$ assigns pattern to class c_i if $d_i(\mathbf{x}) > d_j(\mathbf{x}) \quad \forall j \neq i$

12.3 Optimum statistical classifier - Derivation of the Bayes classifier

- To achieve the optimality of Bayes decision functions, we must know
 - the probability density functions of the patterns in each class
 - the probability of occurrence of each class
- Discussion: What are limitations?

12.3 Optimum statistical classifier - Bayes classifier for Gaussian pattern classes

- Consider n -D problem with two pattern classes ($N_c = 2$) governed by Gaussian densities with means $\mathbf{m}_1$, $\mathbf{m}_2$ and standard deviations σ_1 , σ_2
The decision function becomes

$$d_j(\mathbf{x}) = \frac{1}{(2\pi)^{n/2} |\mathbf{C}_j|^{1/2}} e^{-\frac{1}{2} (\mathbf{x} - \mathbf{m}_j)^T \mathbf{C}_j^{-1} (\mathbf{x} - \mathbf{m}_j)} P(c_j)$$

$$\mathbf{C}_j = \mathbb{E}_j \left\{ (\mathbf{x} - \mathbf{m}_j)(\mathbf{x} - \mathbf{m}_j)^T \right\} = \frac{1}{n_j} \sum_{\mathbf{x} \in c_j} \mathbf{x} \mathbf{x}^T - \mathbf{m}_j \mathbf{m}_j^T$$

$$\mathbf{m}_j = \mathbb{E}_j \{ \mathbf{x} \} = \frac{1}{n_j} \sum_{\mathbf{x} \in c_j} \mathbf{x}$$

n_j is the number of sample
pattern vectors from class c_j

12.3 Optimum statistical classifier - Bayes classifier for Gaussian pattern classes

- The natural logarithm of this decision function

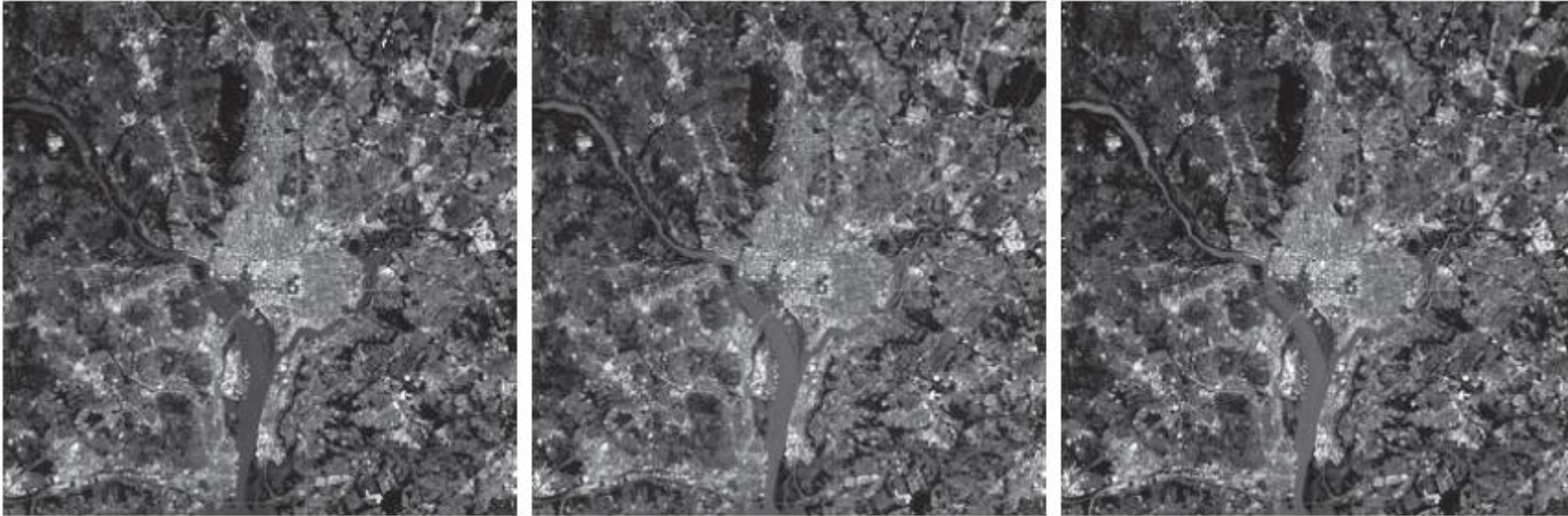
$$\begin{aligned}d_j(\mathbf{x}) &= \ln [p(\mathbf{x}|c_j)P(c_j)] = \ln [p(\mathbf{x}|c_j)] + \ln [P(c_j)] \\&= \ln P(c_j) - \frac{n}{2} \ln(2\pi) - \frac{1}{2} \ln |\mathbf{C}_j| - \frac{1}{2} [(\mathbf{x} - \mathbf{m}_j)^T \mathbf{C}_j^{-1} (\mathbf{x} - \mathbf{m}_j)]\end{aligned}$$

- The term $(n/2)\ln(2\pi)$ is the same for all classes, so it can be eliminated

$$d_j(\mathbf{x}) = \ln P(c_j) - \frac{1}{2} \ln |\mathbf{C}_j| - \frac{1}{2} [(\mathbf{x} - \mathbf{m}_j)^T \mathbf{C}_j^{-1} (\mathbf{x} - \mathbf{m}_j)]$$

The decision functions are hyperquadrics
(quadratic functions in n -dimensional space)!

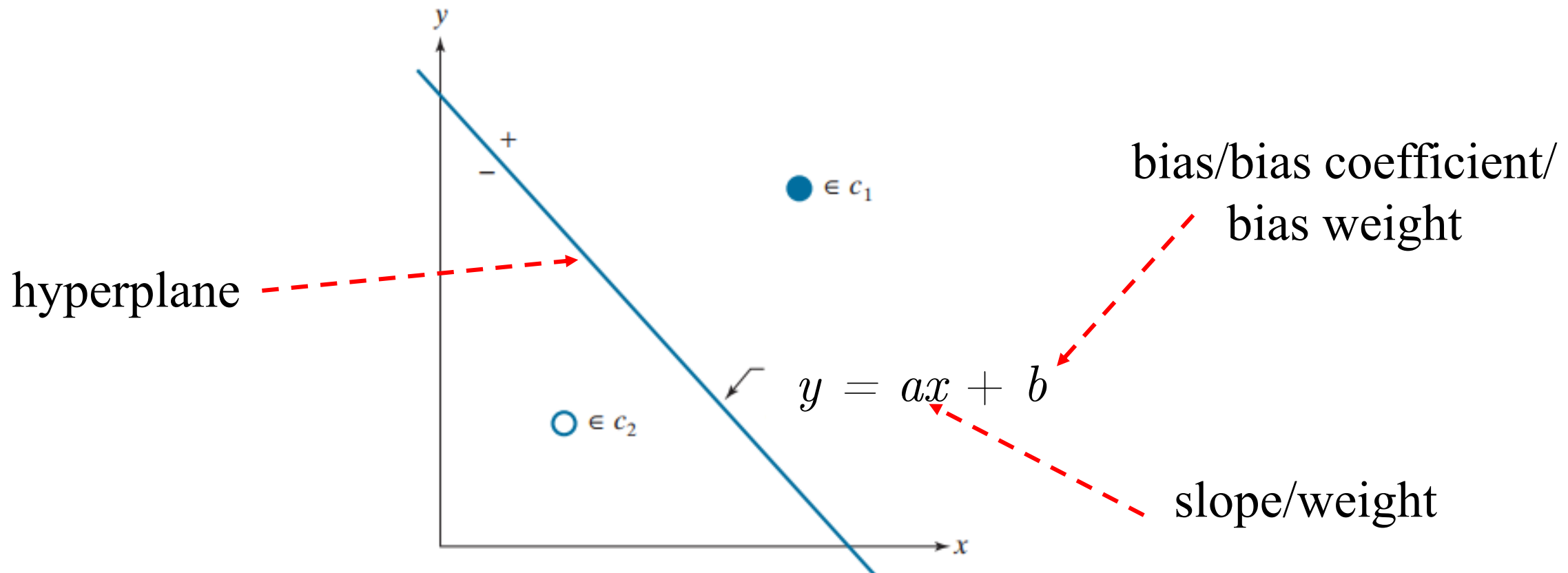
12.3 Optimum statistical classifier - Bayes classifier for Gaussian pattern classes



Training Patterns						Test Patterns					
Class	No. of Samples	Classified into Class			% Correct	Class	No. of Samples	Classified into Class			% Correct
		1	2	3				1	2	3	
1	484	482	2	0	99.6	1	483	478	3	2	98.9
2	933	0	885	48	94.9	2	932	0	880	52	94.4
3	483	0	19	464	96.1	3	482	0	16	466	96.7

12.4 Neural network and Deep learning based classifiers

- The perceptron: A single perceptron unit learns a linear boundary between two linearly separable pattern classes



12.4 Neural network and Deep learning based classifiers

- The hyperplane in n dimensions

$$\sum_{i=1}^n w_i x_i + w_{n+1} = 0 \text{ or } \mathbf{w}^T \mathbf{x} + w_{n+1} = 0$$

- For 2 classes, we want to find a set of weights with the property

$$\mathbf{w}^T \mathbf{x} + w_{n+1} = \begin{cases} > 0 & \text{if } \mathbf{x} \in c_1 \\ < 0 & \text{if } \mathbf{x} \in c_2 \end{cases}$$

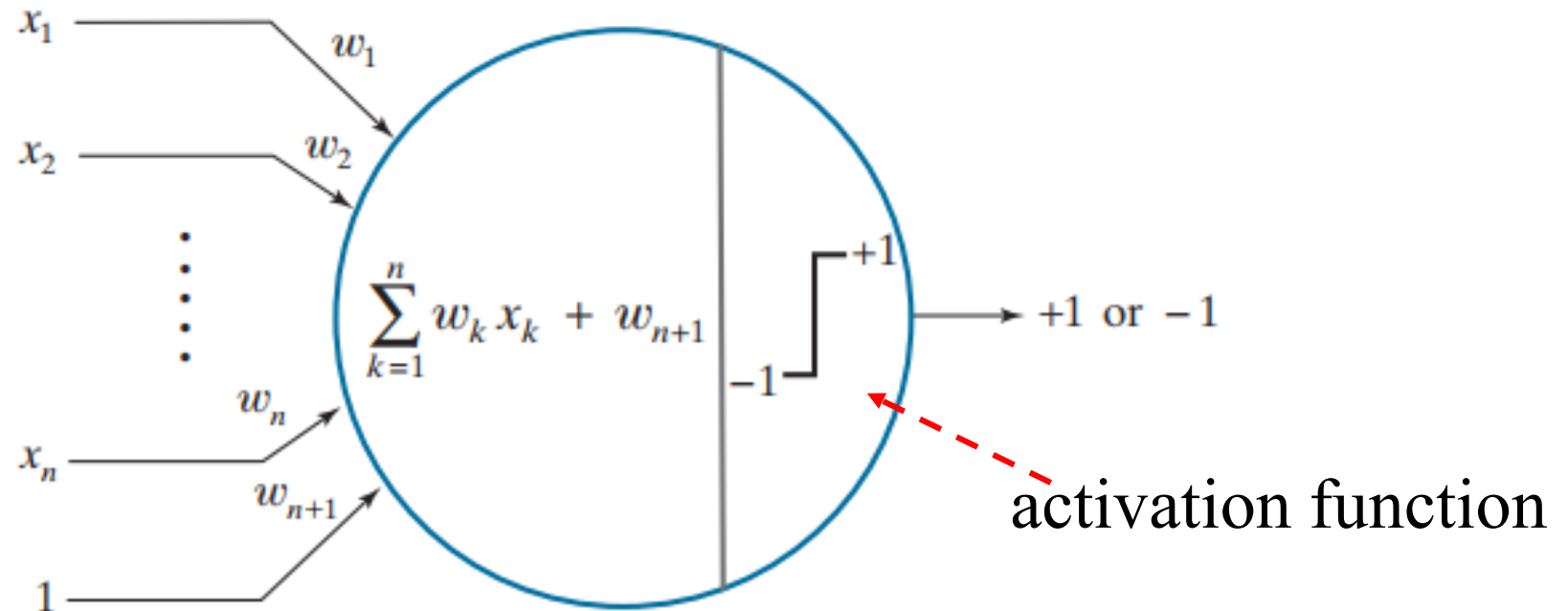
12.4 Neural network and Deep learning based classifiers

- The perceptron algorithm.
- Let $\alpha > 0$ be *correction increment/learning increment/learning rate*, $\mathbf{w}(1)$ be a vector with arbitrary values, and $w_{n+1}(1)$ be an arbitrary constant
- Loop for $k = 2, 3, \dots$
 1. If $\mathbf{x}(k) \in c_1$ and $\mathbf{w}^T(k)\mathbf{x}(k) + w_{n+1}(k) \leq 0$
$$\mathbf{w}(k+1) = \mathbf{w}(k) + \alpha\mathbf{x}(k); w_{n+1}(k+1) = w_{n+1}(k) + \alpha$$
 2. If $\mathbf{x}(k) \in c_2$ and $\mathbf{w}^T(k)\mathbf{x}(k) + w_{n+1}(k) \geq 0$
$$\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha\mathbf{x}(k); w_{n+1}(k+1) = w_{n+1}(k) - \alpha$$
 3. Otherwise $\mathbf{w}(k+1) = \mathbf{w}(k); w_{n+1}(k+1) = w_{n+1}(k)$

12.4 Neural network and Deep learning based classifiers

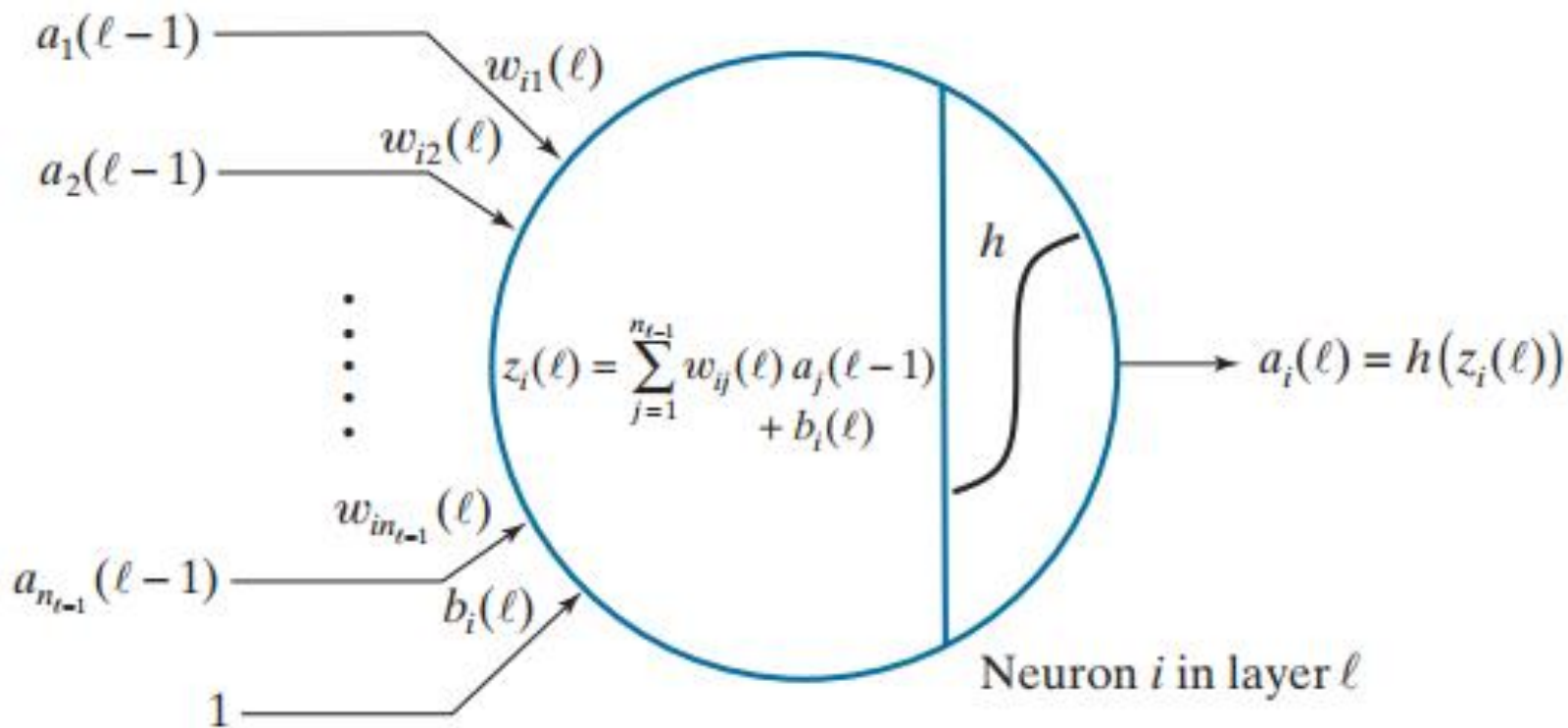
- The *perceptron convergence theorem* states that the algorithm is guaranteed to converge to a solution (i.e., a separating hyperplane) in a finite number of steps if the two pattern classes are linearly separable

schematic of a perceptron



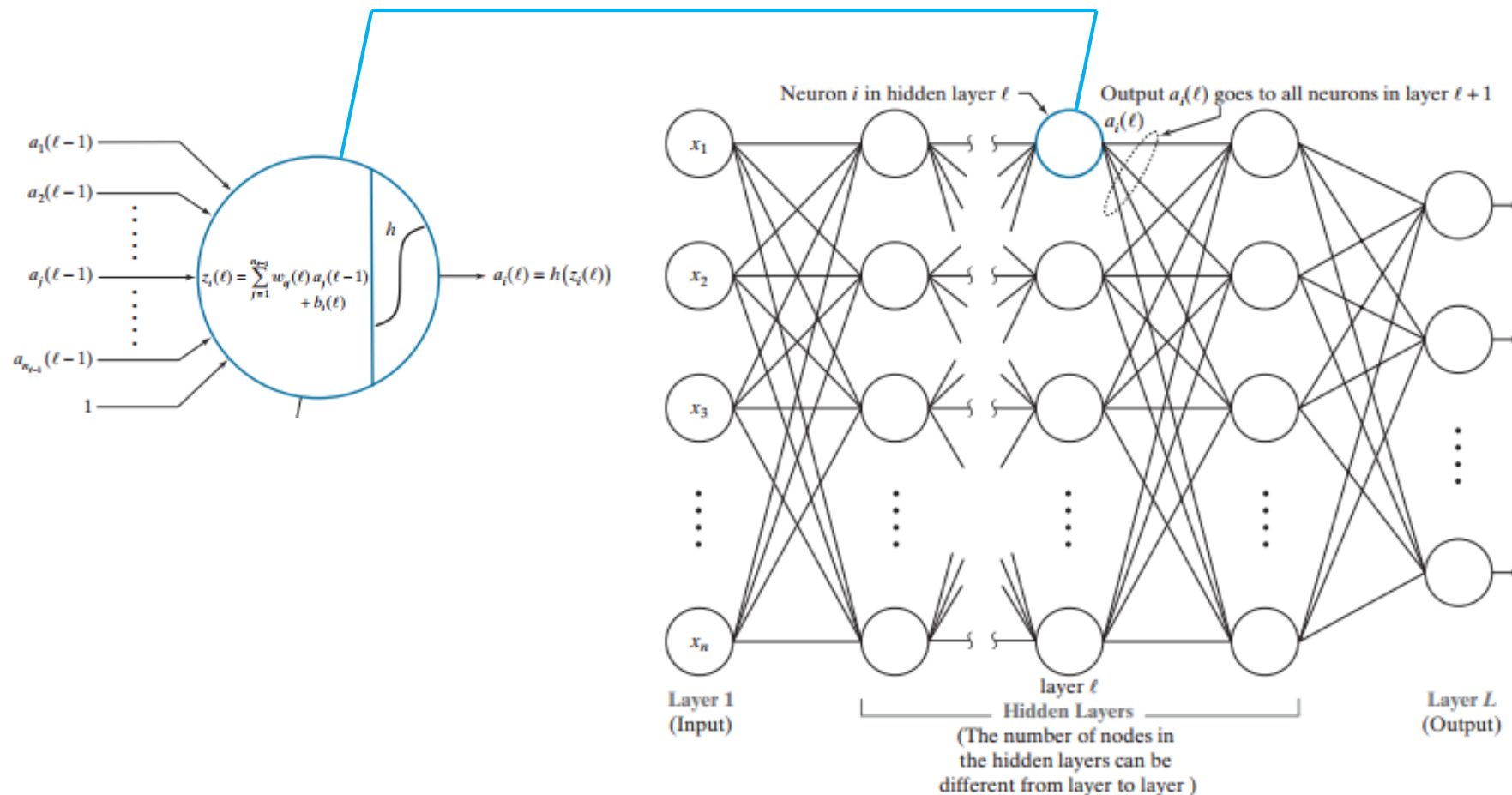
12.4 Neural network and Deep learning based classifiers

- Model of an artificial neuron in layer l



12.4 Neural network and Deep learning based classifiers

- Neural network model

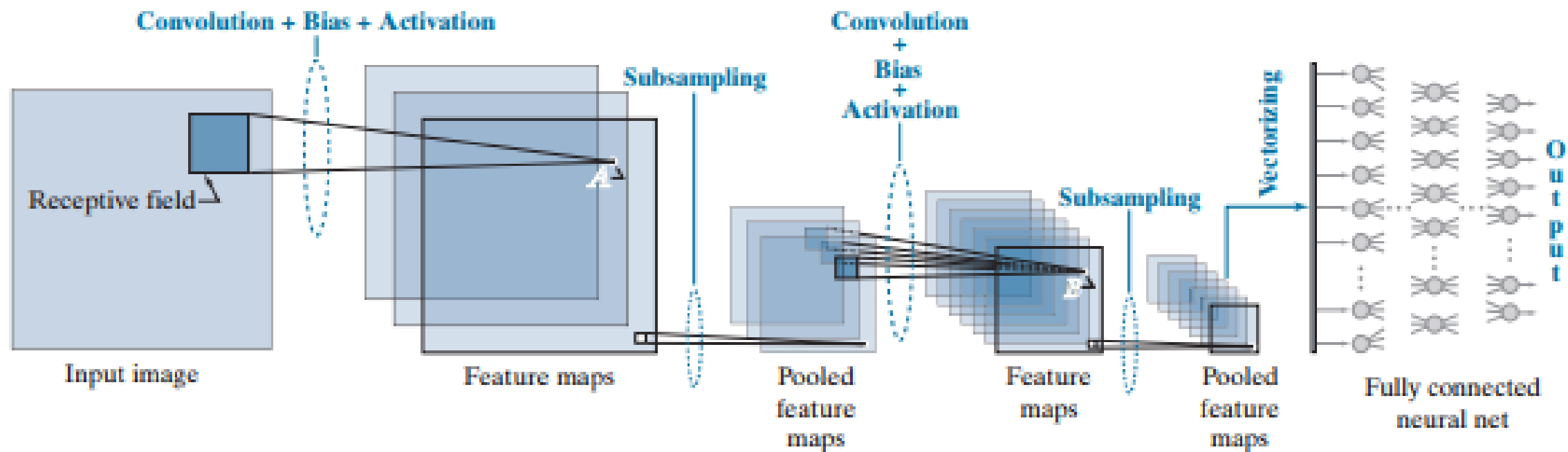


12.4 Neural network and Deep learning based classifiers

- Forwardpropagation through a feedforward neural network
 - A *forward pass* through a neural network maps the input layer (i.e., values of x) to the output layer. The values in the output layer are used for determining the class of an input vector
- Backpropagation to train deep neural networks
 - (1) inputting the pattern vectors;
 - (2) a forward pass through the network to classify all the patterns of the training set and determine the classification error;
 - (3) a backward (backpropagation) pass that feeds the output error back through the network to compute the changes required to update the parameters; and
 - (4) updating the weights and biases in the network

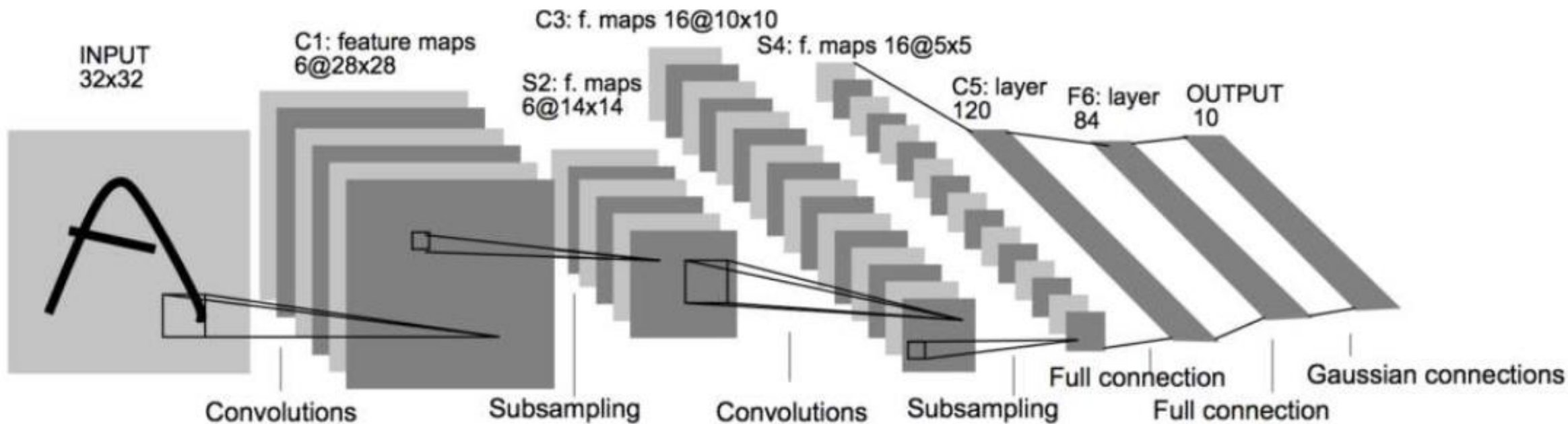
12.5 Deep convolutional neural network based classifiers

- Deep Convolutional Neural Networks



12.5 Deep convolutional neural network based classifiers

- Architecture of the LeNet-5 Convolutional Neural Network

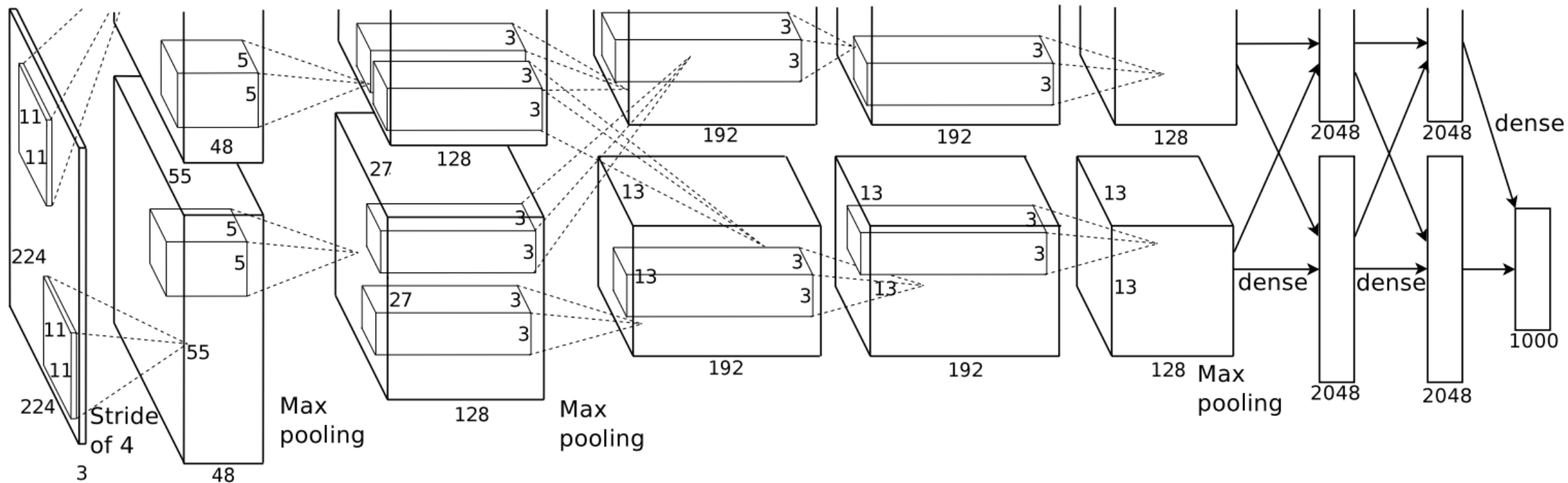


12.5 Deep convolutional neural network based classifiers

- Architecture of the LeNet-5 Convolutional Neural Network
 - Fixed-sized input images.
 - Group convolutional and pooling layers into blocks.
 - Repetition of convolutional-pooling blocks in the architecture.
 - Increase in the number of filters with the depth of the network.
 - Distinct feature extraction and classifier parts of the architecture.

12.5 Deep convolutional neural network based classifiers

- Architecture of the AlexNet Convolutional Neural Network

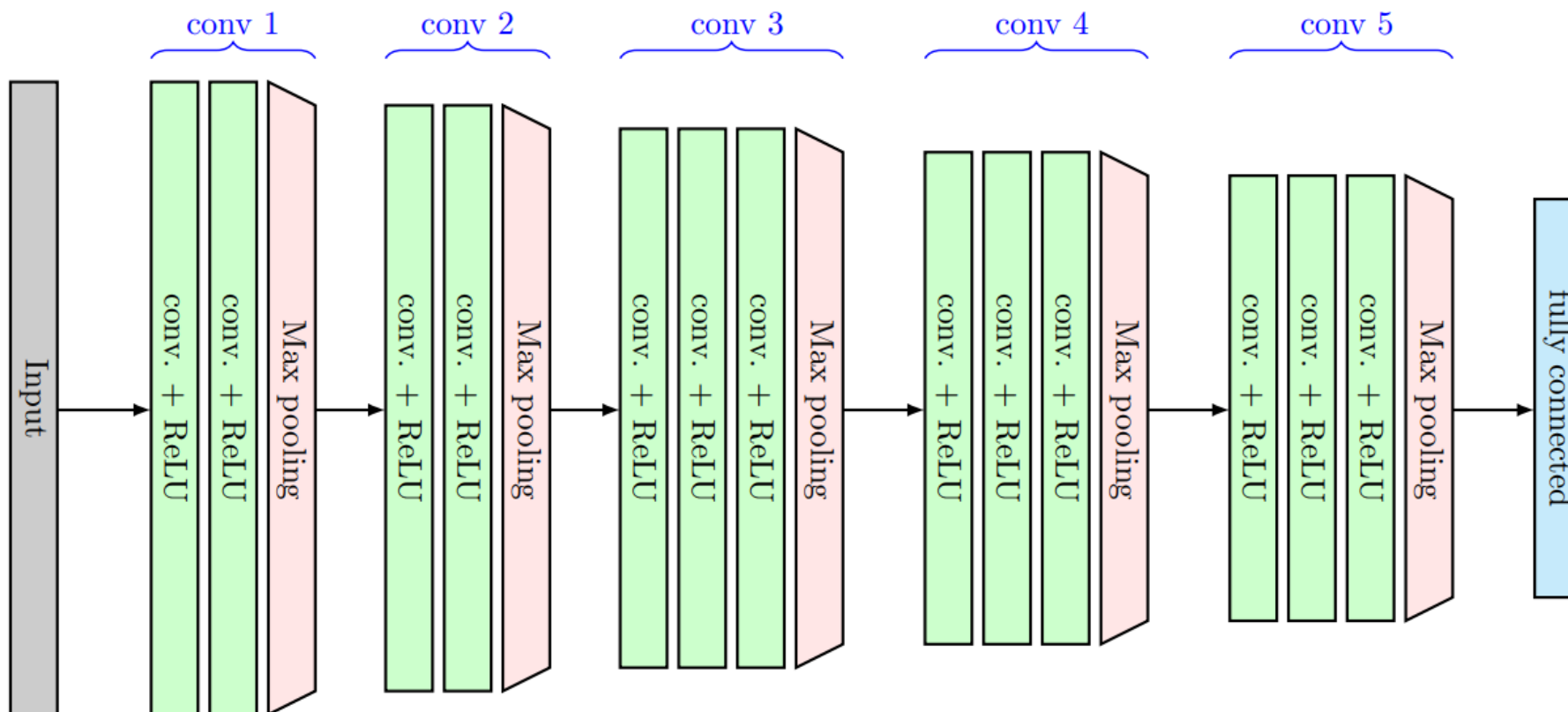


12.5 Deep convolutional neural network based classifiers

- Architecture of the AlexNet Convolutional Neural Network
 - Use of the ReLU activation function after convolutional layers and softmax for the output layer.
 - Use of Max Pooling instead of Average Pooling.
 - Use of Dropout regularization between the fully connected layers.
 - Pattern of a convolutional layer fed directly to another convolutional layer.
 - Use of Data Augmentation.

12.5 Deep convolutional neural network based classifiers

- Architecture of the VGG16 Convolutional Neural Network

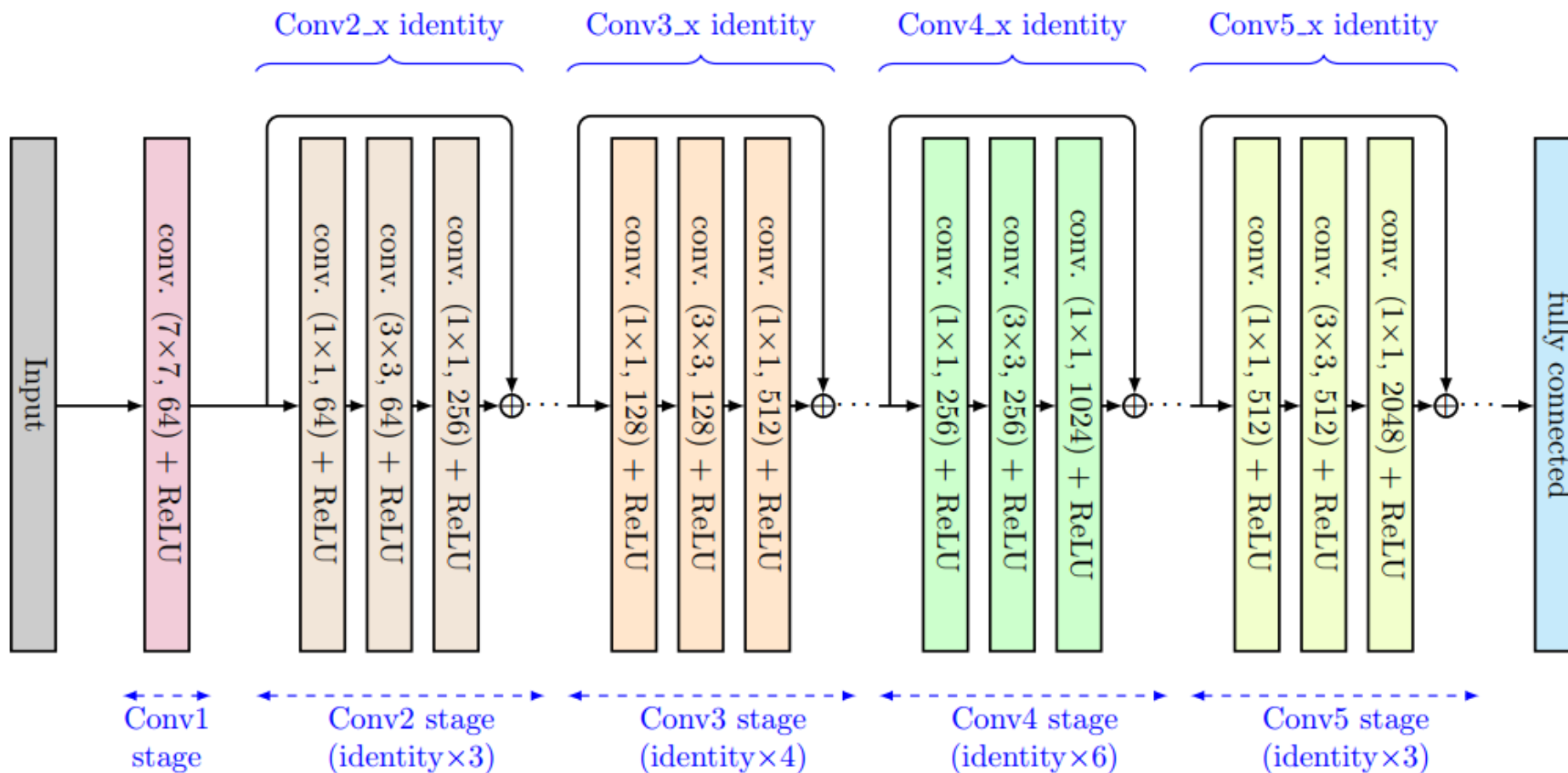


12.5 Deep convolutional neural network based classifiers

- Architecture of the VGG16 Convolutional Neural Network
 - Use of very small convolutional filters, e.g. 3×3 and 1×1 with a stride of one.
 - Use of max pooling with a size of 2×2 and a stride of the same dimensions.
 - The importance of stacking convolutional layers together before using a pooling layer to define a block.
 - Dramatic repetition of the convolutional-pooling block pattern.
 - Development of very deep (16 and 19 layer) models.

12.5 Deep convolutional neural network based classifiers

- Architecture of the Residual Network 50



12.5 Deep convolutional neural network based classifiers

- Architecture of the Residual Network 50
 - Use of shortcut connections.
 - Development and repetition of the residual blocks.
 - Development of very deep (upto 152-layer) models.

Summary

- Fundamentals
- Patterns and pattern classes
- Pattern classification by prototype matching
- Optimum statistical classifier
- Neural network and Deep learning based classifiers
- Deep convolutional neural network based classifiers